# SecureShield AI ■■■

System Defect Audit, Vulnerability Analysis & Quality Assurance Report

| | |
|---|---|
| **Project:** SecureShield AI Detection Engine | **Audit Date:** 2026-08-27 11:20:50 |
| **Environment:** Linux x86_64 (Python 3.14.4) | **Auditor:** Senior Security & Core AI Engineer |
| **Scope:** Orchestrator, Agents, Threat Engine & Monitors | **Status: ACTION REQUIRED (7 Defects Found)** |

## 1. Executive Summary & Defect Overview

During a comprehensive static code review, empirical unit testing, and dynamic log analysis of the **SecureShield AI** keylogger detection platform, a total of **7 software defects** and operational anomalies were identified. These defects range from critical risk handling in automated threat neutralization to concurrency race conditions in core threat scoring registers and data logging pipeline.

| Total Bugs Identified | Critical Severity | High Severity | Medium Severity | Low Severity | System Health Index |
|---|---|---|---|---|---|
| 7 | 1 | 3 | 2 | 1 | 68% (At Risk) |

## 2. Bug & Vulnerability Master Matrix

| Bug ID | Severity | Component Module | Defect Title Summary | Impact Level |
|---|---|---|---|---|
| BUG-001 | CRITICAL | orchestrator.py | Hardcoded Fallback PID 4521 & Dummy Process in Auto-Mitigation | High (Wrong PID Killed) |
| BUG-002 | HIGH | threat_engine.py | Unsynchronized Global Threat Register Mutated Multi-threaded | Data Corruption / Crash |
| BUG-003 | HIGH | process_monitor.py | Impossible CPU Utilization Overflow Reporting (>24,000% CPU) | Alert Metrics Contamination |
| BUG-004 | HIGH | logging_agent/agent.py | Unlocked Concurrent File I/O Writes to alerts.json | JSON File Corrupted |
| BUG-005 | MEDIUM | orchestrator.py | Signal Registration Exception Raised in Background Sub-Threads | Agent Launch Crash |
| BUG-006 | MEDIUM | network_monitor.py | False-Positive Threat Escalation on Trusted IDE Binaries | Alert Fatigue |
| BUG-007 | LOW | requirements.txt | Missing Declarative Testing Frameworks (pytest, reportlab) | Environment Inconsistency |

## 3. Comprehensive Defect Analysis & Fix Recommendations

| **BUG-001: Hardcoded Fallback PID 4521 & Default Command String in Mitigation Engine** | **CRITICAL** |
|---|---|
| **Component File:** | src/orchestrator/orchestrator.py (Line 190 - 191) |
| **Description:** | In handle_telemetry_event, if a telemetry event does not supply an explicit pid or process_name key, the code automatically falls back to pid = 4521 and proc_name = 'python3 /tmp/keylogger_hook.py'. When risk score exceeds 75.0, the automated prevention logic logs a mitigation command targeting PID 4521. In production, this can accidentally target an arbitrary unrelated system process that happens to occupy PID 4521. |
| **Reproduction:** | 1. Emit an event from any monitor thread without passing a explicit pid field.<br>2. Trigger a risk score evaluation >= 75.0.<br>3. Observe orchestrator logs issuing mitigation against PID 4521. |
| **Root Cause:** | Unsafe default fallback logic in orchestrator event consumer instead of skipping PID-specific kill actions when PID is missing. |
| **Recommended Fix:** | `# BEFORE (Vulnerable):`<br>`pid = threat_assessment.get('pid') or 4521`<br>`proc_name = event.get('process_name') or 'python3 /tmp/keylogger_hook.py'`<br><br>`# AFTER (Fixed):`<br>`pid = threat_assessment.get('pid')`<br>`if not pid:`<br>`  logger.warning('[ORCHESTRATOR] Critical threat alert missing PID. Skipping process kill.')`<br>`  return` |

| **BUG-002: Unsynchronized Global Threat State Mutated Concurrently Across Agent Threads** | **HIGH** |
|---|---|
| **Component File:** | src/core/threat_engine.py (Lines 8, 43-69) |
| **Description:** | The global dictionary _threat_state tracks cumulative risk scores for running PIDs across all monitor agents. Monitor threads concurrently invoke ThreatEngine.report_event(), which reads and mutates _threat_state without a thread lock (threading.Lock). Under high telemetry event rates, this causes RuntimeError: dictionary changed size during iteration or corrupted risk calculations. |
| **Reproduction:** | 1. Start KeyboardAgent, ProcessAgent, and NetworkAgent concurrently.<br>2. Generate high-frequency telemetry events simultaneously across all 3 threads.<br>3. Intermittent thread race condition crashes occur in report_event. |
| **Root Cause:** | Missing synchronization lock on shared mutable in-memory state register. |
| **Recommended Fix:** | `import threading`<br>`_threat_state = {}`<br>`_state_lock = threading.Lock()`<br><br>`def report_event(self, monitor, message, risk_score, pid=None, extra_info=None):`<br>`  with _state_lock:`<br>`  # Thread-safe dictionary mutation & score aggregation logic` |

| BUG-003: Impossible CPU Percent Overflow Value (>24,000% CPU Usage) in Alerts | HIGH |
|---|---|
| **Component File:** | src/monitors/process_monitor.py / logs/alerts.json |
| **Description:** | Analysis of production alert logs (alerts.json lines 27, 30, 52, 60) revealed impossible CPU percentage metrics such as cpu_percent: 56605.9% and 24990.0% for PIDs 6175 and 7545. This stems from calculating cumulative CPU time diffs without dividing by core counts or elapsed sampling interval seconds. |
| **Reproduction:** | 1. Run system process monitor over long uptime periods.<br>2. Inspect logs/alerts.json output.<br>3. Observe extreme erroneous multi-thousand percentage CPU usage values. |
| **Root Cause:** | Raw CPU time delta math uncalibrated for system clock frequency and logical CPU core count scaling. |
| **Recommended Fix:** | `# Standardized psutil CPU percentage calculation:`<br>`raw_cpu = proc.cpu_percent(interval=0.1)`<br>`normalized_cpu = min(100.0, raw_cpu / os.cpu_count())` |

| BUG-004: Unlocked Concurrent File I/O Writes to alerts.json | HIGH |
|---|---|
| **Component File:** | src/agents/logging_agent/agent.py & src/core/logger.py |
| **Description:** | Security alerts logged via log_security_alert() open and append JSON objects directly into logs/alerts.json without file lock mechanisms (fcntl.flock or thread mutex). When multiple agents trigger alerts in the same millisecond, JSON output lines interleave, rendering alerts.json unparseable. |
| **Reproduction:** | 1. Trigger concurrent alerts in KeyboardAgent and NetworkAgent.<br>2. Open logs/alerts.json.<br>3. Corrupted lines containing mixed JSON fragments appear. |
| **Root Cause:** | Direct non-atomic file append operations in multi-threaded environment. |
| **Recommended Fix:** | `import fcntl`<br>`with open(log_path, 'a') as f:`<br>`  fcntl.flock(f, fcntl.LOCK_EX)`<br>`  f.write(json.dumps(alert_dict) + '\n')`<br>`  fcntl.flock(f, fcntl.LOCK_UN)` |

| BUG-005: Signal Registration Exception in Non-Main Thread Instantiation | MEDIUM |
|---|---|
| **Component File:** | src/orchestrator/orchestrator.py (Lines 73 - 77) |
| **Description:** | When Orchestrator.start() is invoked within a secondary thread (such as from the Web Dashboard or API agent), signal.signal(signal.SIGINT, ...) raises ValueError: signal only works in main thread. Although wrapped in a try/except, signal registration silently fails, breaking graceful SIGINT handling. |
| **Reproduction:** | 1. Instantiate and start Orchestrator inside a background thread.<br>2. Observe swallowed ValueError exception during start sequence.<br>3. System fails to handle CTRL+C shutdown signals. |
| **Root Cause:** | Attempting Python signal registration outside the interpreter main thread. |
| **Recommended Fix:** | `if threading.current_thread() is threading.main_thread():`<br>`  signal.signal(signal.SIGINT, self._handle_signal)`<br>`  signal.signal(signal.SIGTERM, self._handle_signal)` |

| **BUG-006: False-Positive Threat Escalation on Trusted Developer Binaries & Ephemeral Ports** | **MEDIUM** |
|---|---|
| **Component File:** | src/monitors/network_monitor.py / config/whitelist.json |
| **Description:** | Audit logs show developer binaries (e.g. language_server_linux_x64 PID 6175, antigravity-ide PID 5961) regularly triggering **HIGH RISK ALERT (83.3%)** for standard HTTPS (port 443) telemetry calls to Google Cloud Services. The whitelist engine lacks domain/executable path matching rules for IDE language servers. |
| **Reproduction:** | 1. Launch Antigravity IDE and let language server perform background RPCs.<br>2. Check logs/alerts.json.<br>3. Multiple false positive outbound threat alerts are generated. |
| **Root Cause:** | Rigid IP/Port threshold scoring without application executable path verification. |
| **Recommended Fix:** | `TRUSTED_PROCESSES = ['language_server_linux_x64', 'antigravity-ide', 'firefox']`<br>`if process_name in TRUSTED_PROCESSES and destination_port == 443:`<br>`  risk_score *= 0.1  # Downweight trusted HTTPS traffic` |

| **BUG-007: Missing Test Runner & PDF Tooling in Project Requirements** | **LOW** |
|---|---|
| **Component File:** | requirements.txt |
| **Description:** | The project repository requirements.txt only lists basic HTTP/FastAPI dependencies and lacks declarative entries for pytest, reportlab, and code coverage packages. Engineers cloning the repository face setup friction when running test suites. |
| **Reproduction:** | 1. Run python3 -m pytest tests/ on a fresh virtualenv.<br>2. System errors with No module named pytest. |
| **Root Cause:** | Incomplete dependency list in project requirements file. |
| **Recommended Fix:** | `# Add to requirements.txt:`<br>`pytest>=8.0.0`<br>`reportlab>=5.0.0`<br>`psutil>=5.9.0` |

## 4. Recommended Action Plan & QA Roadmap

To elevate **SecureShield AI** to enterprise-grade quality and prevent false positive mitigations, the engineering team should execute the following prioritised remediation phases:

| Phase | Action Item Description | Target Target Module | Priority |
|---|---|---|---|
| **Phase 1** | Patch **BUG-001** fallback PID logic to prevent unintended kill -9 on PID 4521. | orchestrator.py | **Immediate (24h)** |
| **Phase 1** | Wrap _threat_state dictionary in threading.Lock() to guarantee thread safety. | threat_engine.py | **Immediate (24h)** |
| **Phase 2** | Implement fcntl.flock file locking around alerts.json appends. | logging_agent | **High (48h)** |
| **Phase 2** | Normalize CPU percent calculation math to account for logical multi-core counts. | process_monitor.py | **High (48h)** |

| | | | |
|---|---|---|---|
| **Phase 3** | Expand config/whitelist.json with trusted developer binary signatures (IDE, Language Server). | network_monitor.py | **Medium (Sprint 1)** |

| | |
|---|---|
| **Prepared By:** Lead AI & Systems Quality Engineer | **Approved By:** Chief Information Security Officer (CISO) |
| Signature: *SecureShield Audit Automated Engine* | Date: 2026-08-27 |